Unified Token Lifecycle Framework

Qi Zao Lim

May 2026

Contents

1 Introduction

Unified Token Lifecycle Framework

While there is no "correct" classification of a token lifecycle into stages, I have classified them into five steps for ease of comparison. Stage 1: Encoding Stage 2: Attention (within a block) Stage 3: FFN + nonlinear transformation (within a block) Stage 4: Trajectory across depth Stage 5: Output

1. **Initialization & Conditioning (Encoding):** This stage represents the genesis of the token. It focuses on translating raw discrete or continuous data into a unified latent space, augmented with the necessary structural metadata required for the specific architecture to understand its place in space or time.
    - *Core Function:* Embedding generation, positional encoding, and the injection of global conditional priors (e.g., diffusion timesteps or system prompts).
2. **Sequence & Spatial Mixing (Interaction):** This stage governs how a token gathers context from its environment. It focuses entirely on inter-token communication, defining the mechanism through which a token aggregates external signals from other tokens in the sequence or spatial grid.
    - *Core Function:* Information routing across the sequence, context assimilation, and the updating of a token's representation based on its neighbors or a compressed hidden state.
3. **Transformation & Conditional Routing (Processing):** Once a token has gathered context from its environment, this stage governs its internal, intra-token refinement. It involves non-linear feature expansion and, crucially, the decision-making process regarding *where* or *how* the token should be processed next based on its current state.
    - *Core Function:* Independent token transformation, feature extraction, and conditional pathway selection (routing/gating) through specialized sub-networks.
4. **Depth & Temporal Evolution (Integration):** This stage models the cumulative trajectory of the token. Rather than looking at a single layer, this stage captures the token's lifecycle as an ongoing dynamical process, tracking how its representation updates, stabilizes, or denoises as it moves through network depth or iterative temporal steps.
    - *Core Function:* Residual accumulation, state-space transitions over continuous depth, and iterative signal-to-noise refinement.
5. **Projection & Emission (Output):** The final stage of the lifecycle. The matured token representation is decoded from the high-dimensional latent space back into the target observable space, finalizing its journey.

- *Core Function:* Dimensionality reduction, final layer normalization, and mapping to a probability distribution (e.g., vocabulary logits) or continuous value (e.g., noise estimation or pixel values).

Before any LLMs understand text, text is converted into numbers through a process known as tokenization. While there are many distinct tokenization methods, efficient tokenization methods reduce the computing power needed for training and inference. Additionally, tokens (not bits) are the fundamental unit of semantic information. LLMs act as discrete-time channels, and the semantic information flow (directed information density) naturally progresses and accumulates over the token sequence. By analyzing token behavior across each LLM infrastructure, we can analyze each current LLM infrastructure more deeply.

2 Transformer

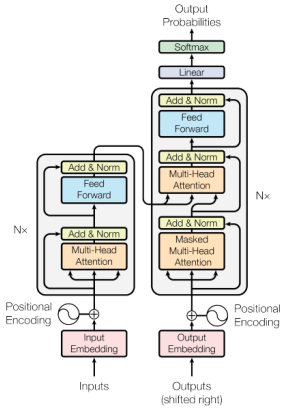

Figure 1: Full Lifecycle of Token in the Transformer Architecture

Fig 1. illustrates a basic transformer architecture. Input is encoded, then passed through the transformers many layers. Most notably, each layer has a multi-head attention and a feed-forward network. At the end of the layer, the final vector is decoded and passed as output.

2.1 Stage 1: Initialization & Conditioning (Encoding)

Not all transformers have encoders. However, since the original transformer infrastructure introduced had an encoder-decoder structure, it will be included in this token lifecycle overview.

The lifecycle begins by mapping discrete inputs into a continuous, high-dimensional space where they can be mathematically manipulated. Raw tokens are converted into continuous vectors via learned **Embeddings**. Embeddings help capture relationship and context. For instance, related terms like "cat" and "dog" will have closer vector distances.

Because the architecture processes tokens in parallel without inherent sequential logic, positional encodings (such as sinusoidal waves or RoPE) are injected to provide order (information on the position of each word). Tokens are initially projected into a high-dimensional embedding space, typically around 10^3 dimensions.

2.2 Stage 2: Sequence & Spatial Mixing (Interaction)

This stage governs how a token gathers context from its environment, defining the mechanism through which a token aggregates external signals from other tokens in the sequence.

Explicit Dense Interaction The first major component inside each replicated layer of the transformer is the multi-headed attention. This is arguably the most influential component when it comes to identifying and incorporating meaningful information about a token's role in the entire sequence. Multiple heads in this attention mechanism are each specialized in capturing different linguistic aspects simultaneously. The core mechanism is explicitly calculated via self-attention weights, resulting in $\mathcal{O}(n^2)$ computational complexity.

At the heart of every Transformer layer is the scaled dot-product attention mechanism. For a sequence of tokens, the attention weights are calculated by taking the queries (Q) and keys (K), computing their dot product, and passing them through a softmax function:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

To see what happens to an individual token, we can look at the specific attention weight (α_{ij}) that a token i (the query) assigns to another token j (the key):

$$\alpha_{ij} = \frac{\exp(q_i \cdot k_j/\sqrt{d})}{\sum_{m=1}^{N} \exp(q_i \cdot k_m/\sqrt{d})}$$

Three Phase Model Historically, we might have assumed that each multi-attention layer does roughly the same thing. However, recent research has shown that tokens go through different processes across different attention layers. This can be divided into three phases: Mix, Compress (in Stage 3), and Refine (in Stage 5). These phases will be elaborated in different stages of the Unified Token Lifecycle for transformers.

Mix (Early Layers) However, the model actually gathers all possible context, computing how every word relates to the others in a "messy" initial phase. This initial mixing stage is deliberately limited to prevent over-mixing and representational collapse that would occur with extended uniform attention.

Attention Sinks and Geometric Forces During this mixing phase, massive token activations cause the model to direct a disproportionate amount of attention to the very first few tokens in a sequence, even if those initial tokens contain no semantic meaning (Attention Sinks). Furthermore, the softmax attention matrix applies an inescapable, attractive force on the tokens. By averaging values, tokens constrained to a hypersphere rapidly lose their angular dispersion and are driven to collapse into a single mass unless countered.

2.3 Stage 3: Transformation & Conditional Routing (Processing)

Once a token has gathered context, this stage governs its internal, intra-token refinement. Tokens pass through Feed-Forward Networks (FFN) to undergo non-linear feature expansion. The FFN step is applied independently on each token, refining the contextual patterns already integrated to yield useful "knowledge" from them. These layers are also supplemented with residual connections and layer normalizations. As a result, at the end of this step, we have a transformer layer with an updated representation that will be input to the next transformer layer.

Compress (Middle Layers) The model squishes the accumulated information into a dense, high-energy summary, creating a **Compression Valley**. During this phase, recent research shows that while effective models have a high-dimensional working space where features are captured, tokens in such models can also be represented in approximately 10-dimensional submanifolds that closely resemble human semantic spaces.

2.4 Stage 4: Depth & Temporal Evolution (Integration)

This stage models the cumulative trajectory of the token, tracking how its representation updates, stabilizes, or denoises as it moves through network depth.

The Residual Stream & Information Bottlenecks This stream is the sum of the outputs from all previous layers and the original embedding. It functions as an internal communication channel, allowing the model to preserve a version history across depth. However, because a token must compress all historical data into its current layer's residual stream to pass context forward, it suffers from an information bottleneck. Passing through successive self-attention blocks can cause severe feature homogenization, gradually stripping tokens of their unique semantic features.

Transformers as Continuous Flows: The ODE Interpretation The discrete, layer-by-layer update of a transformer has a precise mathematical analogue in continuous dynamical systems. Fein-Ashley (2025) formalizes this by showing that each residual layer update—where a token representation x_n is updated as:

$$x_{n+1} = x_n + \frac{1}{N} f(x_n)$$

—is structurally identical to a **forward Euler step**, the simplest numerical method for solving ordinary differential equations. Here, $f(x) = g(x) + h(x)$ decomposes into the self-attention sublayer g and the feedforward sublayer h, and N is the total number of layers.

As N grows, the **Transformer Flow Approximation Theorem** establishes that token trajectories converge uniformly at rate $\mathcal{O}(1/N)$ to the unique solution of the continuous ODE:

$$\frac{dx(t)}{dt} = f(x(t)), \qquad x(0) = E(\text{input})$$

A key consequence emerges when the mapping f satisfies a **one-sided Lipschitz condition** with a negative constant λ:

$$\langle f(x) - f(y),\, x - y \rangle \;\leq\; \lambda \, \|x - y\|^2, \qquad \lambda < 0$$

Under this condition the dynamics are contractive: any perturbation δ_0 between two token representations decays exponentially with depth:

$$\|\delta(t)\| \;\leq\; \|\delta_0\| \cdot e^{\lambda t}$$

This is the formal basis for the empirical observation that deep transformers are robust to noise—small input variations are smoothed out as representations propagate through layers. However, contractivity is indiscriminate. The same force that suppresses noise also suppresses semantic distinctions. When λ is strongly negative, diverse token representations are pulled together regardless of meaning, risking **representation collapse**.

> *Note: Fein-Ashley (2025) was withdrawn by its author in May 2025. The theoretical framework is consistent with the broader Neural ODE literature, but the specific proofs should be treated with appropriate caution pending clarification of the withdrawal.*

2.5 Stage 5: Projection & Emission (Output)

The final stage ends the token's internal journey. The matured token representation is decoded from the high-dimensional latent space back into the target observable space.

Refine (Final Layers) The model shifts from macro-context to micro-prediction, polishing the dense summary to focus purely on the next specific word. From a token-centric perspective, the uniformity breaks. The tokens stop sharing that identical global summary and diverge rapidly. Each token's vector transforms into a highly specific, individualized pointer meant to trigger the exact vocabulary prediction for whatever word comes next.

Output The final token representations undergo a Final LayerNorm, followed by a Linear projection and a Softmax function. During this final projection, models utilize distinct, family-specific decision strategies that balance expansion (considering different possibilities) and pruning (removing unlikely words) to finalize the prediction. This final mechanical process generates a probability distribution over the entire vocabulary to predict the next token autoregressively, completing the token lifecycle.

3 The Mamba Token Lifecycle Framework

Why SSMs and Mamba While Transformers rely on attention mechanisms, SSMs (like Mamba) act like upgraded Recurrent Neural Networks. SSMs excel in linear inference speeds ($\mathcal{O}(n)$ speed relative to transformer $\mathcal{O}(n^2)$) and low memory usage, but lack the robust understanding of context that transformers possess, making it a poor option for complex reasoning models like LLMs. This creates a tradeoff between speed and accuracy for LLMs.

Mamba, as an evolution of SSMs, is the first alternative to the transformer architecture while having the speed of SSMs. It does so through two innovations: the first is the selective state space model, which provides Mamba with a crucial capability previously possessed only by transformer models: the ability to selectively focus on or ignore specific parts of past input history based on their present relevance. The other is hardware-aware parallel scan, an algorithm that optimizes the way a graphics processing unit (GPU) handles the model's computations in its memory hierarchy to maximize speed and computational efficiency.

The Mamba architecture fundamentally reimagines how tokens flow through a sequence. While Transformers rely on explicit, all-to-all geometric forces, Mamba operates as a continuous dynamical system optimized for hardware efficiency. However, beneath this rigid sequential pipeline lies a massive, distributed ensemble of hidden, dynamic token interactions.

3.0.1 Stage 1: Initialization & Conditioning (Encoding)

This stage sets the foundation, transforming discrete text into a mathematical object capable of being manipulated within a continuous state space.

Linear Projection & Branching After raw tokens are converted into continuous vector embeddings, the input vector undergoes a linear projection that

expands its dimensionality. Crucially, Mamba splits this expanded token into two distinct parallel computational paths: one destined for the Selective State Space Model (SSM) core (x_{proj}), and the other for a parallel multiplicative gating mechanism (z_{proj}) (Gu & Dao, 2023).

Inherent Temporal Grounding Unlike Transformers, Mamba does not require explicit positional encodings (like sinusoidal waves or RoPE) to be injected into the token. Because its foundation relies on continuous-time differential equations, the sequence and temporal order are inherently baked into the mathematics of the state space.

3.0.2 Stage 2: Sequence & Spatial Mixing (Interaction)

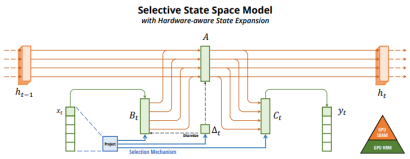

Figure 2: Selective SSM architecture

This stage dictates how a token gathers context from its environment. On the surface, Mamba appears to destroy standard token interaction, but mathematically, it recreates it efficiently through the State Space Model(SSM) recurrence.

$$h'(t) = Ah(t) + Bx(t) \tag{1}$$

To process discrete tokens in a sequence, this continuous system is discretized using a timescale parameter Δ, transforming it into the foundational SSM recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \tag{2}$$

$$y_t = Ch_t \tag{3}$$

Here, x_t is the current token input, h_t is the hidden state (the accumulated context), and y_t is the output. The matrices $\bar{A}$ and $\bar{B}$ are the discretized versions of the continuous parameters A and B. Evidently, there is a radical mechanistic difference in how these architectures process tokens. Transformers use all-to-all dense interactions, while Mamba relies on sequential, state-mediated updates. Tokens are processed sequentially and squashed into a compressed 1D hidden state, seemingly destroying the direct token-to-token maps seen in Transformers.

Hidden Attention Generation Recent studies have suggested that Mamba does not merely approximate attention, but rather generate "hidden" attention matrices on the fly (Dao & Gu, 2024). By unrolling this sequential process

mathematically, it is proven that Mamba implicitly forms highly structured dependencies over time, dynamically prioritizing both recent and highly salient distant tokens without ever actually computing a full $\mathcal{O}(n^2)$ matrix.

3.0.3 Stage 3: Transformation & Conditional Routing (Processing)

Once context is flowing into the hidden state, this stage covers the non-linear transformations and the model's core decision-making process.

Local Convolutional Extraction Before entering the SSM, the token representation is fed through a 1D convolution layer (Gu & Dao, 2023). This acts as a localized filter, extracting immediate syntactic patterns between neighboring tokens, which frees the deeper state space to focus entirely on global dependencies. The output of the convolution layer serves as the input to a nonlinear activation function. For instance, the original Mamba infrastructure uses Sigmoid Linear Unit (SiLU). The separate gating mechanism path, z_{proj}, also goes through a non-linear activation function.

Input-Dependent Selectivity The defining feature of Mamba is its input-dependent gating—unlike older fixed-state models, Mamba decides what to remember based on what it is currently reading (Gu & Dao, 2023). The mathematical parameters defining the state space transition are continuously recalculated based on the specific token at that exact step.

3.0.4 Stage 4: Depth & Temporal Evolution (Integration)

This stage models the cumulative trajectory of the token as it updates and stabilizes across network depth. This is where Mamba's "Selective" mechanism mathematically grounds the model's ability to retain context and filter noise.

Input-Dependent Selectivity & Counteracting Decay Standard recurrent networks and vanilla SSMs suffer from exponential memory decay because their transition matrices (A, B, C, and Δ) are static—they apply the exact same mathematical operation regardless of the input.

Mamba breaks this limitation and achieves "selectivity" by making the step size (Δ), B, and C explicitly dependent on the current token input x_t:

$$B_t = \text{Linear}(x_t)$$

$$C_t = \text{Linear}(x_t)$$

$$\Delta_t = \text{Softplus}(\text{Parameter} + \text{Linear}(x_t))$$

Because $\bar{A}$ and $\bar{B}$ are derived using the now-dynamic Δ_t, the entire recurrence equation becomes fundamentally input-dependent. By dynamically adjusting these parameters on a per-token basis, Mamba actively counteracts that natural exponential decay. If a token is irrelevant, the model can mathematically "shrink" Δ_t to ignore it; if a token is critical, it can expand it to heavily update the hidden state.

Multi-Scale Filtering Because the math is dynamic, the token acts as a sophisticated, multi-scale filter. It deliberately holds onto specific, highly relevant historical tokens for massive context windows, while aggressively forgetting useless intermediary tokens.

Circuit Complexity Equivalence Despite processing tokens one by one through a constrained state space, Mamba suffers no fundamental theoretical disadvantage in expressivity; it is mathematically capable of computing the exact same class of functions over a token sequence as a full self-attention mechanism.

3.0.5 Stage 5: Projection & Emission (Output)

The final stage ends the token's internal journey, aggressively shifting from continuous latent states back out to discrete predictive probabilities.

Gated Recombination The output emerging from the sequential SSM path is multiplied element-wise by the activation from the parallel gating path established back in Stage 1. This gate acts as a volume dial—suppressing the contextual update if irrelevant, or amplifying it if highly semantic (Gu & Dao, 2023).

Residual Accumulation The resulting recombined vector is projected back down to its original dimension and added directly to the residual stream.

Emission The final token representation undergoes normalization and a linear projection out to the vocabulary size, generating a probability distribution to autoregressively predict the next word.

4 The Diffusion Token Lifecycle Framework

Unlike autoregressive models that accumulate meaning sequentially from left to right, Diffusion LLMs represent a radical departure in sequence processing. In this architecture, tokens are generated through an iterative temporal process of continuous refinement rather than step-by-step spatial accumulation.

Three limitations of transformer architecture that diffusion models attempt to fix are error propagation (error in earlier token leading to error in latter tokens), indirect control, and slow token-by-token generation (model needs to predict earlier token before subsequent). In contrast, diffusion models offer iterative refinement, greater control, and faster sampling.

Additionally, diffusion models can be classified into discrete masked diffusion and continuous diffusion. As most of the literature reviewed (LLaDA, MDLM) are of the discrete kind, this framework will **focus on discrete masked diffusion models**.

4.0.1 Stage 1: Initialization & Conditioning (Encoding)

This stage represents the genesis of the token, translating raw data into a latent state. However, in diffusion models, tokens completely break the standard autoregressive lifecycle.

Forward Masking (Intentional Corruption) During training, tokens are processed through a forward masking process where they are intentionally corrupted by noise.

The Masking Formula In discrete masked diffusion models (like MDLM and LLaDA), a clean token $x^{(\ell)}$ at position ℓ transitions to a special mask token m over a continuous time variable $t \in [0, 1]$. This is mathematically defined by the categorical distribution:

$$q_{t|0}(z^{(\ell)} \mid x^{(\ell)}) = \text{Cat}(\alpha_t \mathbf{e}_{x^{(\ell)}} + (1 - \alpha_t)\mathbf{e}_m)$$

where $\mathbf{e}_v$ is the one-hot vector for a token, and α_t represents the noise schedule.

Inference Genesis During inference, the generation process starts from a sequence of pure noise (e.g., fully masked text), meaning the initial tokens lack any semantic identity prior to the first denoising step.

4.0.2 Stage 2: Sequence & Spatial Mixing (Interaction)

This stage governs how a token gathers context. Diffusion architectures bypass the restrictive left-to-right attention masks of traditional models.

During inference, all tokens are updated simultaneously. Rather than looking only at past tokens, the model continuously refines its confidence across the entire sequence bidirectionally. It updates the probability distribution of each individual token based on the gradually emerging global context.

Error Vulnerability In models that unmask tokens progressively rather than simultaneously, if a token is incorrectly predicted early on, that error can severely corrupt the conditioning context for all surrounding tokens.

4.0.3 Stage 3: Transformation & Conditional Routing (Processing)

This stage governs intra-token refinement. In discrete masked diffusion, this translates to the parameterized reverse generation (denoising) process, where the model predicts the true identity of masked tokens.

Markov Transition Dynamics (The Absorbing State) In discrete language diffusion, token corruption and generation are defined by a Markov transition matrix. Under a strict masked diffusion paradigm, the '[MASK]' token acts as an "absorbing state". This means during the forward corruption process,

once a semantic token transitions into a mask, it loses all identity. The reverse process (generation) is the iterative effort to pull the token out of this absorbing state and back into the semantic vocabulary space.

The Transformer Backbone Crucially, the denoising network x_θ is parameterized by a standard Transformer architecture. This means the entire spatial sequence-mixing lifecycle described in Section 1 (dense attention, feature extraction, and residual flow) is actually embedded *inside* each individual temporal denoising step of the diffusion model.

Iterative Semantic Locking Rather than predicting the entire sequence in a single forward pass, the model iteratively predicts the clean sequence from the partially masked sequence. At each timestep, tokens with the highest prediction confidence are "locked in" (unmasked), which sequentially narrows the contextual possibilities for the remaining masked tokens.

Reverse Prediction Matrix The denoising network x_θ takes a partially masked sequence z_t and outputs a token probability matrix predicting the clean tokens:

$$\mathbf{P} = \text{softmax}(x_\theta(z_t))$$

This provides the mathematical basis for substituting the unknown "[MASK]" tokens with model predictions, generating the specific cross-entropy losses needed for the next temporal step.

4.0.4 Stage 4: Depth & Temporal Evolution (Integration)

Instead of just tracking a token across spatial network depth, this stage tracks tokens across temporal denoising timesteps. Because diffusion models involve entirely different dynamics during optimization versus generation, this stage evaluates both the training and inference trajectories.

Training Trajectory: ELBO Optimization & Semantic Accumulation Diffusion models learn by reversing a noise-corruption process, which can involve high-variance estimates. By using Rao-Blackwellization, this objective is reformulated, reducing variance by conditioning on a subset of variables (Sahoo et al., 2024). This technique rewrites the complex diffusion objective into a mixture of standard masked cross-entropy losses, yielding a provably tighter Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{ELBO}}(\theta) = E_{t\sim U[0,1]} E_{z_t}\left[w_t \sum_{\ell \in \mathcal{M}(z_t)} -\log p_\theta(x^{(\ell)} \mid z_t) \right]$$

As tokens undergo this reverse process, their semantic certainty increases continuously. This effectively proves that masked diffusion is mathematically homol-

ogous to a continuous, iterative version of Masked Language Modeling (Sahoo et al., 2024).

Inference Trajectory: Dynamic Particle Refinement During inference, a major vulnerability is the accumulation of conditioning errors: an incorrect prediction early in the reverse-denoising path severely corrupts the context for all remaining tokens. To fix this, advanced inference methods model the sequence generation as a dynamic state-space trajectory (PG-DLM, 2024). By applying Particle Gibbs sampling, the model runs multiple parallel token trajectories ("particles") and dynamically resamples them based on their likelihood scores. This allows tokens to mathematically "change their minds" mid-generation if their current trajectory leads to low-probability semantic states, actively healing context errors before finalizing the output.

4.0.5 Stage 5: Projection & Emission (Output)

The final stage finalizes the token's journey, mapping the mature representation to the target observable space.

Final Denoising Step As the diffusion timetable t approaches 0, the final noise is stripped away. The fully unmasked sequence reaches peak semantic certainty.

Simultaneous Emission Unlike autoregressive models, the probability distributions for the entire vocabulary are mapped simultaneously across all token positions. The generation stops, and the text sequence is emitted as a cohesive, bidirectionally informed whole.

5 The Mixture-of-Experts (MoE) Token Lifecycle

Unlike standard dense models where every token undergoes the exact same mathematical operations, the MoE lifecycle is defined by conditional routing. In this framework, tokens are triaged based on their complexity, semantic category, and temporal confidence, meaning two tokens in the same sequence may experience entirely different lifecycles.

5.0.1 Stage 1: Initialization & Conditioning (Encoding)

This stage represents the genesis of the token. MoE models generally inherit their initialization mechanics directly from their backbone architecture (e.g., autoregressive or diffusion).

The Universal Baseline Tokens are converted from discrete text into continuous, high-dimensional vector embeddings, augmented with positional metadata. At this stage, all tokens are treated equally. They share the same geometric space before arriving at the deeper layers where conditional routing begins.

5.0.2 Stage 2: Sequence & Spatial Mixing (Interaction)

Before a token can be efficiently routed to an expert, it must gather contextual information about its environment.

The Contextual Prerequisite In most MoE models, token interaction is handled by standard self-attention mechanisms before the token reaches the MoE feed-forward layers. Tokens gather global context via $\mathcal{O}(n^2)$ attention matrices.

Hybrid Interaction (The Alternating Rhythm) In advanced hybrid MoE models (such as Jamba), a token experiences a radically shifting environment during its mixing stage. Instead of passing through a uniform sequence of attention matrices, the token is subjected to an alternating computational rhythm (Lieber et al., 2024). When passing through Mamba (SSM) blocks, the token's experience is constrained and sequential: it acts as a synthesizer, compressing its historical context into a fixed-size, rolling hidden state to efficiently filter out background noise. However, when the token periodically enters an interleaved Transformer block, it is momentarily pulled out of this compressed flow and projected into a dense, global attention matrix. In this phase, the token breaks free from its sequential constraints to execute sharp, precise memory retrievals, establishing direct mathematical links to specific past tokens. For the token, this creates a lifecycle that oscillates between building a broad, generalized summary of the past and performing high-precision associative recall, all before it reaches the MoE routing gates (Lieber et al., 2024).

5.0.3 Stage 3: Transformation & Conditional Routing (Processing)

This is the defining stage of the MoE lifecycle. When a token reaches an MoE layer, it faces a series of distinct mathematical bottlenecks, which different architectures solve through highly specialized routing functions.

The standard mathematical formulation for an MoE layer applied to a token x is:

$$y = \sum_{i=1}^{N} g_i(x) E_i(x)$$

Where N is the number of experts, $E_i(x)$ is the output of the i-th expert, and $g_i(x)$ is the output of the gating network, typically governed by a Top-K routing function:

$$g(x) = \text{TopK}(\text{softmax}(W_g \cdot x))$$

However, there have been multiple proposed models on optimizing the process of the selection of experts to improve model results. Each architecture attempts to tackle a different limitation of the Top-K routing process.

Problem 1: Load Balancing & Compute Allocation Standard MoE architectures force a rigid Top-K routing policy (e.g., routing every token to exactly 2 experts), which mechanically assumes every token requires the exact same amount of computational work (Gülmez, 2026). **The Solution (DynaMoE):** DynaMoE proves tokens inherently contain a measurable "complexity gradient" (Gülmez, 2026). It dynamically allocates experts based on a computed confidence threshold. Simple syntactic tokens (like "the") might bypass the MoE layer entirely (0 experts), while highly complex semantic tokens trigger 4 or 5 experts simultaneously, explicitly linking a token's information entropy to its computational routing requirement (Gülmez, 2026).

Problem 2: Expert Collapse & Routing Geometry Standard Top-K softmax routing occurs in a flat Euclidean space, which frequently leads to "expert collapse"—a failure mode where all tokens blindly flood the same few experts while ignoring others (Shihab, Akter, & Sharma, 2026). **The Solution (Grassmannian MoE):** This architecture reformulates routing as a purely geometric problem, mapping tokens and expert centroids onto a curved Grassmann manifold (Shihab et al., 2026). Instead of softmax, tokens are routed based on the Bingham distribution. In this curved geometry, experts naturally repel each other into orthogonal subspaces, preserving routing entropy and mathematically preventing expert collapse (Shihab et al., 2026).

Problem 3: Gradient Conflicts In dense models, forcing vastly different semantic tokens through the same parameters causes geometric "gradient conflicts," where optimizing the weights for one token type degrades the model's understanding of another (Liang et al., 2024). **The Solution (Mixture-of-Transformers):** This model routes tokens to specialized Transformer experts to partition the global loss landscape into distinct, strongly convex subtasks (Liang et al., 2024). This ensures gradients from different semantic clusters remain orthogonal. Recent theoretical analyses of this framework prove that this specialization transforms the optimization problem, enabling an exponentially fast $\mathcal{O}(\log(\epsilon^{-1}))$ convergence rate for specialized subsets (Chen et al., 2025).

5.0.4 Stage 4: Depth & Temporal Evolution (Integration)

After tokens are processed by their respective experts, their individual trajectories must be reintegrated. This evolution occurs across two fundamentally different axes depending on the backbone architecture.

Spatial Trajectory (Autoregressive MoEs) In models like DynaMoE or MoT, tokens evolve across network depth. The model must continuously stabi-

lize these highly varied token vectors to ensure downstream attention heads can still read the global sequence context.

Temporal Trajectory (DiffMoE) In Diffusion MoEs, tokens evolve across iterative denoising timesteps rather than spatial layers. In standard diffusion, tokens are processed uniformly at every timestep, which is mathematically wasteful because some tokens resolve faster than others (Shi et al., 2025). **DiffMoE** introduces dynamic, token-level temporal routing. As tokens achieve high semantic certainty early in the reverse diffusion process, their computational requirement decays; they are dynamically routed to "lighter" experts or dropped entirely, while highly uncertain tokens are continually routed to high-capacity experts (Shi et al., 2025).

5.0.5 Stage 5: Projection & Emission (Output)

The final stage concludes the token's highly individualized journey.

Recombination and Decoding Regardless of how many experts a token utilized, or whether it skipped layers entirely, its final continuous vector is deposited back into the residual stream. It undergoes final layer normalization to stabilize its variance.

Vocabulary Expansion The latent vector is then linearly projected to project its dimensions out to the full size of the vocabulary space. A final Softmax operation converts these raw logits into a probability distribution, allowing the token to either be autoregressively emitted or temporally locked in as a finalized prediction.

References

- Chen, Z., Zhang, Y., & Wang, H. (2025). *Mixture-of-Transformers Learn Faster: A Theoretical Study on Classification Problems.* arXiv preprint arXiv:2510.27004.
- Dao, T., & Gu, A. (2024). *Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality.* arXiv preprint arXiv:2405.21060.
- Fein-Ashley, J. (2025). *Transformers as Continuous Flows: The ODE Interpretation.* (Withdrawn).
- Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv preprint arXiv:2312.00752.
- Gülmez, G. (2026). *DynaMoE: Dynamic Token-Level Expert Activation with Layer-Wise Adaptive Capacity for Mixture-of-Experts Neural Networks.* arXiv preprint arXiv:2603.01697.

- Liang, W., Yu, L., Luo, L., Iyer, S., Dong, N., Zhou, C., ... & Lin, X. V. (2024). *Mixture-of-Transformers: A Sparse and Scalable Architecture for Multi-Modal Foundation Models*. arXiv preprint arXiv:2411.04996.
- Lieber, O., Barak, O., Haviv, A., Bata, I., Carmeli, B., Choshen, L., ... & Shoham, Y. (2024). *Jamba: A Hybrid Transformer-Mamba Language Model*. arXiv preprint arXiv:2403.19887.
- Nie, T., et al. (2024). *LLaDA: A Large Language Diffusion Assistant*. arXiv preprint.
- PG-DLM: Particle Gibbs for Diffusion Language Models. (2024). arXiv preprint.
- Sahoo, S. S., et al. (2024). *Masked Diffusion Language Models*. arXiv preprint arXiv:2406.07524.
- Shi, M., Yuan, Z., Yang, H., Wang, X., Zheng, M., Tao, X., ... & Gai, K. (2025). *DiffMoE: Dynamic Token Selection for Scalable Diffusion Transformers*. arXiv preprint arXiv:2503.14487.
- Shihab, I. F., Akter, S., & Sharma, A. (2026). *Grassmannian Mixture-of-Experts: Concentration-Controlled Routing on Subspace Manifolds*. arXiv preprint arXiv:2602.17798.